



Department of Electrical and Computer  
Engineering  
North South University

**Directed Research**  
**FPCE\*: An Optimized Exact  
Pseudo-Clique Enumerator**

**S M Samiul Islam      ID #2231971042**

**Omar Sadat      ID #2221335042**

Faculty Advisor:  
**Dr. Ahsanur Rahman**  
Associate Professor  
ECE Department

**Spring, 2026**

## APPROVAL

S M Samiul Islam (ID #2231971042), Omar Sadat (ID #2221335042), from the Electrical and Computer Engineering Department of North South University, have worked on the Senior Design Project titled FPCE\*: An Optimized Exact Pseudo-Clique Enumerator under the supervision of Dr. Ahsanur Rahman, for partial fulfillment of the requirement for the degree of Bachelors of Science in Engineering and has been accepted as satisfactory.

Supervisor's Signature

.....

**Dr. Ahsanur Rahman**

**Associate Professor**

Department of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh.

Chairman's Signature

.....

**Dr. Mohammad Abdul Matin**

**Professor**

Department of Electrical and Computer Engineering

North South University

Dhaka, Bangladesh.

## DECLARATION

This is to declare that this project is our original work. No part of this work has been submitted elsewhere partially or fully for the award of any other degree or diploma. All project-related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor. Relevant previous works presented in this report have been properly acknowledged and cited. The plagiarism policy, as stated by the supervisor, has been maintained.

### Students' Names and Signatures

1. S M Samiul Islam

-----

2. Omar Sadat

-----

## ACKNOWLEDGEMENTS

We would like to express our heartfelt gratitude towards our project and research supervisor, Dr. Ahsanur Rahman, Associate Professor, Department of Electrical and Computer Engineering, North South University, Bangladesh, for his invaluable support, precise guidance, and advice pertaining to the experiments, research, and theoretical studies carried out during the course of the current project and also in the preparation of the current report.

Furthermore, we would like to thank the Department of Electrical and Computer Engineering, North South University, Bangladesh for facilitating the research. We would also like to thank Mohammad Shoaib and our friends Atiya Ibnat Tasnim for helping us in this project. We would also like to thank our loved ones for their countless sacrifices and continual support.

# ABSTRACT

Pseudo-cliques (subgraphs with almost all possible edges) have many applications, but their exact enumeration is computationally expensive due to the large search space. Although recent algorithms such as FPCE significantly reduce this cost using structural pruning, their practical performance still depends heavily on implementation choices and data representations. In this work, we present an optimized exact pseudo-clique enumerator based on the DensePCE framework. Our approach integrates FPCE’s pruning techniques—order bound, core-aware edge bound, and Turán-based clique seeding—while employing a Compressed Sparse Row graph layout and intrusive bucket-based degree maintenance for efficient reverse search. We further accelerate enumeration by using the EBBkC algorithm to enumerate mandatory cliques and use them as search seeds. Experiments on large real-world graphs show that our implementation consistently outperforms FPCE, achieving substantial runtime reductions while preserving exactness, demonstrating that careful integration of pruning strategies with dense-optimized data structures leads to significant practical gains.

# Table of Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction .....</b>                                    | <b>1</b>  |
| 1.1      | Background and Motivation.....                               | 1         |
| 1.1.1    | Cliques, Pseudo-Cliques, and Edge Density .....              | 1         |
| 1.1.2    | The Enumeration Problem .....                                | 2         |
| 1.1.3    | Reverse-Search Enumeration.....                              | 3         |
| 1.1.4    | Auxiliary Concepts .....                                     | 5         |
| 1.1.5    | Motivation for DensePCE .....                                | 6         |
| 1.2      | Purpose and Goal of the Project.....                         | 7         |
| 1.2.1    | Purpose .....                                                | 7         |
| 1.2.2    | Goals .....                                                  | 8         |
| 1.2.3    | Scope and Boundaries.....                                    | 10        |
| 1.3      | Organization of the Report .....                             | 10        |
| <b>2</b> | <b>Literature Review .....</b>                               | <b>12</b> |
| 2.1      | Existing Research and Limitations .....                      | 12        |
| 2.1.1    | Limitations of Existing Work.....                            | 15        |
| 2.1.2    | Motivation for This Work.....                                | 16        |
| <b>3</b> | <b>Methodology .....</b>                                     | <b>18</b> |
| 3.1      | System Design .....                                          | 18        |
| 3.1.1    | Methodological Overview .....                                | 18        |
| 3.1.2    | Reverse-Search Design .....                                  | 19        |
| 3.1.3    | Seeded Search Strategy .....                                 | 19        |
| 3.1.4    | Candidate-Pool Optimization Beyond the Baseline Design ..... | 20        |

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| 3.1.5    | Candidate Uncle Inheritance .....                                      | 20        |
| 3.1.6    | Outside-Gap Theorem .....                                              | 22        |
| 3.1.7    | System-Level Workflow .....                                            | 25        |
| 3.2      | Hardware and/or Software Components .....                              | 26        |
| 3.2.1    | Software Components of the Methodology .....                           | 26        |
| 3.2.2    | Graph Storage Component .....                                          | 26        |
| 3.2.3    | State-Maintenance Component .....                                      | 27        |
| 3.2.4    | Intrusive Bucket Component .....                                       | 27        |
| 3.2.5    | Structural Analysis Component .....                                    | 28        |
| 3.2.6    | Seed Generation Component .....                                        | 28        |
| 3.2.7    | Output and Statistics Component .....                                  | 28        |
| 3.3      | Hardware and/or Software Implementation .....                          | 28        |
| 3.3.1    | Feasibility Test During Expansion .....                                | 29        |
| 3.3.2    | Maximality Check .....                                                 | 29        |
| 3.3.3    | Uniqueness-Aware Child Generation .....                                | 30        |
| 3.3.4    | Order-Bound Implementation .....                                       | 30        |
| 3.3.5    | Turán-Based Clique Seeding Implementation .....                        | 31        |
| 3.3.6    | Core-Aware Edge-Bound Pruning .....                                    | 31        |
| 3.3.7    | Deterministic and Reproducible Execution .....                         | 32        |
| 3.3.8    | Rollback-Based Execution Flow .....                                    | 32        |
| 3.3.9    | Implementation Outcome .....                                           | 32        |
| <b>4</b> | <b>Investigation/Experiment, Result, Analysis and Discussion .....</b> | <b>34</b> |
| 4.1      | Investigation and Experimental Setup .....                             | 34        |
| 4.1.1    | Benchmark Graphs and Preprocessing .....                               | 34        |
| 4.1.2    | Experimental Setup .....                                               | 35        |
| 4.1.3    | Evaluation Metrics .....                                               | 36        |
| 4.1.4    | Correctness Verification .....                                         | 36        |
| 4.2      | Results .....                                                          | 37        |
| 4.2.1    | Runtime Comparison Without Turán-Based Seeding .....                   | 37        |

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| 4.2.2    | Runtime Comparison With Full Optimization .....                             | 38        |
| 4.2.3    | Ablation of Candidate Uncle Inheritance and the Outside-Gap<br>Theorem..... | 38        |
| 4.2.4    | Results for Candidate Uncle Inheritance .....                               | 39        |
| 4.2.5    | Results for the Outside-Gap Theorem .....                                   | 40        |
| 4.2.6    | Discussion of the Combined Effect.....                                      | 41        |
| 4.2.7    | Graphical Comparison .....                                                  | 41        |
| 4.3      | Analysis and Discussion .....                                               | 44        |
| 4.3.1    | Interpretation of the Runtime Trends .....                                  | 44        |
| 4.3.2    | Ablation-Based Discussion .....                                             | 45        |
| 4.3.3    | Practical Significance .....                                                | 45        |
| <b>5</b> | <b>Conclusions</b> .....                                                    | <b>47</b> |
| 5.1      | Summary .....                                                               | 47        |
| 5.2      | Limitations .....                                                           | 48        |
| 5.3      | Future Improvement.....                                                     | 50        |



# List of Figures

|     |                                                        |    |
|-----|--------------------------------------------------------|----|
| 4.3 | FPCE versus DensePCE on scale free graphs. ....        | 43 |
| 4.4 | FPCE versus DensePCE on small world graphs. ....       | 43 |
| 4.5 | DensePCE speed up over FPCE on scale free graphs. .... | 43 |
| 4.6 | DensePCE speed up over FPCE on small world graphs..... | 43 |
| 4.1 | FPCE versus DensePCE with full optimization.....       | 43 |
| 4.2 | FPCE versus DensePCE without Turán-based seeding.....  | 43 |

# List of Tables

|     |                                                                                                                                                                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Properties of the real graph datasets used in the experiments. Here, $\text{den}(G)$ denotes graph density and $D$ denotes graph degeneracy. ....                                                                                                                       | 35 |
| 4.2 | Comparison of CPU time between FPCE and DensePCE without Turán-based seeding. The speedup is computed as $T_{\text{FPCE}}/T_{\text{DensePCE}}$ . ....                                                                                                                   | 37 |
| 4.3 | Comparison of CPU time between FPCE and DensePCE with the complete optimization pipeline enabled. The speedup is computed as $T_{\text{FPCE}}/T_{\text{DensePCE}}$ .....                                                                                                | 38 |
| 4.4 | Comparison between the baseline DensePCE configuration and DensePCE with Candidate Uncle Inheritance (CUI). Since the output-dependent graph statistics remain unchanged across the two methods, only runtime, peak memory, and CUI-specific savings are reported. .... | 39 |
| 4.5 | Comparison between DensePCE with Candidate Uncle Inheritance (CUI) and DensePCE with CUI plus the Outside-Gap theorem. As the search space and graph-dependent counts remain unchanged, only runtime, peak memory, and Uncle saves are reported.....                    | 40 |

# Chapter 1 Introduction

## 1.1 Background and Motivation

This section establishes the formal definitions and algorithmic foundations on which DensePCE builds. We first recall the graph-theoretic notions of cliques and pseudo-cliques (Section 1.1.1), then formalize the enumeration problem (Section 1.1.2). We next review the reverse-search framework and the PCE/FPCE line of work (Section 1.1.3), and finally motivate the implementation-level redesign pursued in this paper (Section 1.1.5).

### 1.1.1 Cliques, Pseudo-Cliques, and Edge Density

Let  $G = (V, E)$  be a simple undirected graph with  $n = |V|$  vertices and  $m = |E|$  edges. For a vertex set  $S \subseteq V$ , the *induced subgraph*  $G[S] = (S, E(S))$  contains every edge of  $G$  whose both endpoints belong to  $S$ , i.e.,  $E(S) = \{\{u, v\} \in E : u, v \in S\}$ .

**Definition 1.1 (Clique)** *A vertex set  $S \subseteq V$  is a clique if every pair of distinct vertices in  $S$  is connected by an edge, i.e.,  $|E(S)| = \binom{|S|}{2}$ .*

Cliques capture the tightest possible cohesion: every member interacts with every other member. However, in practice, real-world networks are noisy—edges may be missing due to incomplete observations, measurement errors, or inherent stochasticity [1]. Requiring perfect pairwise connectivity therefore risks excluding meaningful dense communities. This observation motivates the relaxed notion of a *pseudo-clique*.

**Definition 1.2 (Edge density)** For a vertex set  $S \subseteq V$  with  $|S| \geq 2$ , the edge density of  $S$  is

$$\text{den}(S) = \frac{|E(S)|}{\binom{|S|}{2}} = \frac{2|E(S)|}{|S|(|S| - 1)}.$$

By convention,  $\text{den}(S) = 1$  when  $|S| \leq 1$ .

**Definition 1.3 ( $(\ell, \theta)$ -pseudo-clique)** Given a minimum size threshold  $\ell \geq 2$  and a density threshold  $\theta \in (0, 1]$ , a vertex set  $S \subseteq V$  is an  $(\ell, \theta)$ -pseudo-clique if  $|S| \geq \ell$  and  $\text{den}(S) \geq \theta$ .

When  $\theta = 1$ , an  $(\ell, 1)$ -pseudo-clique is simply a clique of size at least  $\ell$ . As  $\theta$  decreases, structures with progressively more missing edges are admitted. The parameters  $\ell$  and  $\theta$  together allow the analyst to control both the minimum community size and the tolerated sparsity, making pseudo-cliques a flexible tool for community detection in biological, social, and information networks [1, 2].

**Definition 1.4 (Maximal  $(\ell, \theta)$ -pseudo-clique)** An  $(\ell, \theta)$ -pseudo-clique  $S$  is maximal if no proper superset  $S' \supsetneq S$  is an  $(\ell, \theta)$ -pseudo-clique.

Maximality ensures that the enumerated structures are not redundant: every maximal pseudo-clique represents a dense community that cannot be further enlarged without violating the density constraint.

### 1.1.2 The Enumeration Problem

The central computational task addressed in this paper is:

**Problem (MAXPSEUDOCliqueENUM).** Given a graph  $G = (V, E)$ , a minimum size  $\ell \geq 2$ , and a density threshold  $\theta \in (0, 1]$ , enumerate all maximal  $(\ell, \theta)$ -pseudo-cliques in  $G$ .

Unlike optimization formulations that seek a single densest subgraph, enumeration aims to output the complete collection of maximal dense substructures. This is desirable in exploratory analyses where the number and distribution of dense communities is itself informative.

**Computational challenges.** Pseudo-clique enumeration is significantly harder than clique enumeration for two interconnected reasons:

1. **Non-hereditary feasibility.** A subset of a pseudo-clique need not be a pseudo-clique (the density may increase or decrease upon removing a vertex), and a superset of a pseudo-clique may again become feasible after temporarily losing feasibility. This non-monotonic behavior prevents the simple monotone pruning used by Bron–Kerbosch-style clique enumeration algorithms [3, 4].
2. **Exponential output size.** The number of maximal pseudo-cliques can be exponential in the number of vertices, even for moderate  $\theta$ . Output-sensitive algorithms, whose running time scales with the number of solutions, are therefore the appropriate complexity target.

### 1.1.3 Reverse-Search Enumeration

The *reverse-search* paradigm, introduced by Avis and Fukuda [5], provides a systematic way to enumerate combinatorial objects by defining a parent function over the solution space. If the parent function induces a rooted tree spanning all solutions, traversing this tree by depth-first search yields a complete enumeration.

#### Uno’s Framework for Dense Subgraphs

Uno [6] adapted reverse search to the enumeration of dense subgraph patterns, including pseudo-cliques. The key idea is to define, for every maximal  $(\ell, \theta)$ -pseudo-clique  $S$  other than the root, a deterministic *parent* pseudo-clique  $\text{par}(S)$  obtained by removing a specific vertex from  $S$  and then extending to maximality. The parent rule must satisfy two properties:

- **Well-definedness:**  $\text{par}(S)$  is uniquely determined and is itself a maximal  $(\ell, \theta)$ -pseudo-clique (or the designated root).
- **Acyclicity:** Repeated application of  $\text{par}(\cdot)$  eventually reaches the root, so the parent relation defines a tree on the solution space.

Given these properties, a depth-first traversal of the implicit parent tree enumerates every maximal pseudo-clique exactly once. Polynomial delay is achieved if each recursive step—computing children and checking feasibility—runs in polynomial time.

### **Pseudo-Clique Enumerator (PCE)**

Building directly on Uno’s framework, the *Pseudo-Clique Enumerator* (PCE) implements reverse-search enumeration for maximal  $(\ell, \theta)$ -pseudo-cliques with polynomial delay per output [6]. At each search node, PCE maintains a current pseudo-clique  $S$ , attempts to add or remove vertices to generate candidate children, checks density feasibility and maximality, and verifies the parent rule to avoid redundant traversals.

While PCE is complete and correct, its practical performance on large real-world graphs is limited. The recursion tree can be very deep, and without additional pruning, vast numbers of intermediate nodes are explored that ultimately yield no new maximal pseudo-cliques.

### **Fast Pseudo-Clique Enumerator (FPCE)**

The *Fast Pseudo-Clique Enumerator* (FPCE) [7] retains the reverse-search skeleton of PCE but introduces three pruning techniques that dramatically reduce the explored search tree:

1. **Degeneracy-based order bound.** By processing vertices in degeneracy order, FPCE derives an upper bound on the size of any pseudo-clique that can be formed from the remaining vertices. Branches exceeding this bound are pruned early.
2. **Turán-theorem-based clique-existence filter.** The Turán theorem gives a lower bound on the number of edges required to guarantee the existence of a clique of size  $r$  within a graph. FPCE uses this to determine a minimum clique size  $R$  that must exist in any feasible pseudo-clique of size  $\ell$  and density  $\theta$ . If no such  $R$ -clique exists in the candidate neighborhood, the branch can be safely pruned.

3. **Core-aware edge bound.** By leveraging the  $k$ -core decomposition of the graph, FPCE computes tighter bounds on the number of edges that a candidate subgraph can attain, enabling further branch elimination.

These pruning strategies are *sound*—they never discard valid maximal pseudo-cliques—and together they yield orders-of-magnitude speedups on large sparse real-world graphs compared to the unpruned PCE baseline.

### 1.1.4 Auxiliary Concepts

Several auxiliary graph-theoretic concepts play important roles in both the pruning strategies and the data-structure design of DensePCE.

**Degeneracy and  $k$ -Core Decomposition.** The *degeneracy* of a graph  $G$  is the smallest integer  $d$  such that every subgraph of  $G$  has a vertex of degree at most  $d$ . Equivalently, the degeneracy equals the maximum core number over all vertices. The  $k$ -core of  $G$  is the maximal subgraph in which every vertex has degree at least  $k$ . The  $k$ -core decomposition can be computed in  $O(n + m)$  time by iteratively peeling minimum-degree vertices [8, 9]. The resulting *degeneracy ordering* processes vertices from lowest to highest core number and provides the foundation for the order-bound pruning in FPCE and DensePCE alike.

**Turán’s Theorem.** Turán’s theorem [10] states that among all graphs on  $n$  vertices that do not contain a complete subgraph  $K_{r+1}$ , the *Turán graph*  $T(n, r)$  has the maximum number of edges:

$$|E(T(n, r))| = \left(1 - \frac{1}{r}\right) \frac{n^2}{2}.$$

Equivalently, any graph on  $n$  vertices with more than  $|E(T(n, r))|$  edges must contain a  $(r+1)$ -clique. In the context of pseudo-clique enumeration, this theorem provides a necessary condition for the existence of cliques of a given size within a candidate subgraph, and is the theoretical basis for the Turán-based clique-existence filter used in FPCE.

**Compressed Sparse Row (CSR) Format.** The *Compressed Sparse Row* (CSR) representation stores only the non-zero structure of a sparse graph using two arrays: an *offset array* of size  $n + 1$  indicating where each vertex’s adjacency list begins and ends, and a *neighbors array* of size  $2m$  storing the endpoints in sorted order. This format provides  $O(1)$  access to any vertex’s degree and  $O(\log \deg(v))$  neighbor lookup via binary search, while achieving excellent cache locality during sequential neighborhood scans—a property exploited heavily in DensePCE’s per-recursion operations.

**Intrusive Bucket-Based Degree Lists.** Maintaining per-vertex degree information during recursive add/remove operations is a bottleneck for reverse-search enumerators. An *intrusive doubly-linked list with bucket indexing* allows a vertex to be moved between degree buckets in  $O(1)$  time upon each edge update. By “intrusive,” we mean that each vertex node contains its own list pointers, eliminating separate memory allocation and improving cache behavior. This structure enables DensePCE to incrementally maintain the degree distribution of the current candidate subgraph at negligible overhead per recursion step.

### 1.1.5 Motivation for DensePCE

While FPCE achieves substantial pruning improvements over PCE, its implementation relies on general-purpose data structures (adjacency-list representations and standard containers) that incur non-trivial constant-factor overheads in per-recursion operations such as neighborhood intersection, degree queries, and vertex addition/removal. On large graphs with millions of edges, these constant factors accumulate over billions of recursive calls and become the dominant cost.

DensePCE is motivated by three complementary observations:

1. **Cache efficiency matters at scale.** Modern CPUs are heavily bandwidth-limited. Replacing pointer-chasing adjacency lists with a flat, sorted CSR layout aligns memory access with hardware prefetching and reduces cache misses during the neighborhood scans that dominate each recursion step.



2. **Incremental bookkeeping can be made  $O(1)$ .** The per-step cost of updating degree information and checking density feasibility can be reduced from  $O(\deg)$  to  $O(1)$  amortized by maintaining intrusive bucket-based degree lists that track the degree distribution of the current candidate set.
3. **Clique seeding can replace expensive tree levels.** The Turán-based filter in FPCE checks whether a required  $R$ -clique *exists* but still begins the reverse-search tree from small initial sets. If we instead *enumerate all  $R$ -cliques* upfront using an efficient clique lister (EBBkC [11]), each clique serves as a *seed* from which the reverse search continues. This effectively prunes the upper levels of the search tree—the levels with the highest branching factor—and amortizes the seed-computation cost over the many pseudo-cliques grown from each seed.

Together, these design choices yield a system that preserves the theoretical guarantees of reverse-search enumeration (completeness, no duplication, polynomial delay) while significantly reducing the wall-clock time on practical instances. DensePCE thus represents an *engineering-driven* contribution: it demonstrates that careful attention to data layout, memory hierarchy, and algorithmic initialization can complement pruning-based improvements and push the practical frontier of exact dense subgraph enumeration.

## 1.2 Purpose and Goal of the Project

### 1.2.1 Purpose

The purpose of this project is to design and implement an optimized exact enumerator for maximal  $(\ell, \theta)$ -pseudo-cliques that achieves substantial wall-clock speedups over the current state-of-the-art algorithm, FPCE, without sacrificing correctness or completeness. While FPCE introduced powerful pruning strategies—order bound, Turán-based clique-existence filtering, and core-aware edge bound—that dramatically

reduce the explored search space, its practical performance remains constrained by per-recursion overhead arising from general-purpose data structures and memory access patterns that do not exploit modern hardware characteristics.

This project addresses this gap by developing **DensePCE**, a system that couples FPCE’s theoretically grounded pruning techniques with implementation-level optimizations specifically engineered for the reverse-search enumeration workload. The central thesis is that *careful attention to data layout, memory hierarchy, and algorithmic initialization can yield performance improvements that are complementary to—and composable with—pruning-based advances*, thereby pushing the practical frontier of exact dense subgraph enumeration on large real-world graphs.

### 1.2.2 Goals

The specific goals of this project are as follows:

1. **Cache-Friendly Graph Representation.** Replace the adjacency-list representation used in prior enumerators with a sorted *Compressed Sparse Row (CSR)* format. The CSR layout stores neighbor arrays contiguously in memory, enabling sequential access patterns that align with hardware prefetching and minimize cache misses during the neighborhood scans that dominate each recursion step. Adjacency queries are supported in  $O(\log d(v))$  time via binary search on sorted neighbor arrays.
2. **Efficient Incremental State Maintenance.** Design an *intrusive bucket-based degree list* structure that tracks, for every vertex, its current degree into the partial pseudo-clique  $P$ . Vertices are organized into doubly-linked lists indexed by degree, allowing  $O(1)$  per-vertex bucket moves upon each add or remove operation. This eliminates the need for repeated linear scans over all vertices when computing the minimum internal degree  $\delta(P)$ , the add-degree threshold  $d_{\min}$ , and the maximality test—operations that are invoked at every recursion node.
3. **Integration of FPCE-Aligned Pruning.** Incorporate the three pruning tech-

niques introduced by FPCE into the DensePCE engine while preserving correct reverse-search semantics:

- *Order bound*: Compute a degeneracy-derived upper bound  $\mu$  on pseudo-clique size and terminate expansion once  $|P| = \mu$ .
- *Core-aware edge bound*: At each recursion node with  $|P| < \ell$ , compute an upper bound on the maximum number of edges achievable at size  $\ell$  using the minimum core number and internal degree of  $P$ , and prune the subtree if the required edge count  $L = \lceil \theta \binom{\ell}{2} \rceil$  is unattainable.
- *Turán-based clique-existence filter*: Derive the minimum clique size  $R$  that every feasible  $(\ell, \theta)$ -pseudo-clique must contain, and use this as a necessary condition during search.

4. **R-Clique Seeding via EBBkC.** Go beyond FPCE’s use of the Turán property as a passive filter by operationalizing it as an active *initialization strategy*. Specifically, enumerate all  $R$ -cliques on the  $(R-1)$ -core of the input graph using the EBBkC exact  $k$ -clique lister as a streaming subroutine. Each enumerated clique is injected into the search state as a *seed*, from which reverse search continues. This effectively bypasses the upper levels of the recursion tree—precisely the levels with the highest branching factor and lowest pruning effectiveness—and amortizes the one-time seed-enumeration cost over the many maximal pseudo-cliques grown from each seed.

5. **Empirical Validation and Ablation.** Conduct a rigorous experimental evaluation on a diverse set of real-world graphs to:

- Verify that DensePCE produces exactly the same set of maximal  $(\ell, \theta)$ -pseudo-cliques as FPCE (correctness check).
- Measure end-to-end CPU time and compute per-instance speedups of DensePCE over FPCE under identical parameter settings.
- Perform ablation studies that isolate the contributions of individual com-

ponents (data-structure improvements alone vs. data structures plus each pruning technique) to understand which optimizations are most impactful and how they interact.

6. **Reproducibility and Modularity.** Deliver a clean, modular C++ implementation with compile-time or runtime switches for each optimization (order bound, edge bound, Turán seeding). This enables future researchers to (a) reproduce all reported results, (b) selectively enable or disable individual components for further ablation, and (c) extend the system with new pruning strategies or alternative seed-generation methods without modifying the core enumeration engine.

### 1.2.3 Scope and Boundaries

To maintain focus, the following aspects are explicitly outside the scope of this project:

- **New pruning theory.** We do not propose novel pruning bounds; instead, we integrate and operationalize existing FPCE bounds within an optimized runtime.
- **Parallel enumeration.** The current implementation is single-threaded. Multi-core parallelization of the reverse-search tree is left as future work.
- **Approximate or heuristic methods.** DensePCE is an *exact* enumerator. We do not compare against approximate dense-subgraph discovery methods, as the output semantics differ.
- **Weighted, directed, or temporal graphs.** The present framework targets simple undirected graphs. Extensions to richer graph models are discussed as future directions.

## 1.3 Organization of the Report

The remainder of this report is organized as follows. Chapter 1 (Introduction) provides the background, motivation, and objectives of the project, along with an overview of the report structure. Chapter 2 (Literature Review) discusses the main ex-

act pseudo-clique enumeration methods, their pruning strategies, and the remaining gap in candidate-generation overhead that motivates this work. Chapter 3 (Methodology) describes the system design of DensePCE, including the baseline optimized enumeration engine, the Candidate Uncle Inheritance rule, and the Outside-Gap theorem. Chapter 4 (Investigation/Experiment, Result, Analysis and Discussion) presents the experimental setup, the DensePCE-versus-FPCE comparison, and the ablation results for Candidate Uncle Inheritance and the Outside-Gap theorem. Finally, Chapter 5 (Conclusions) summarizes the key findings, outlines the limitations of the work, and suggests possible directions for future improvements.

# Chapter 2 Literature Review

## 2.1 Existing Research and Limitations

**Dense subgraph models and early enumeration.** Abello *et al.* [12] formally introduced the  $\gamma$ -quasi-clique model, which requires every vertex in a subgraph of size  $k$  to have at least  $\gamma(k - 1)$  neighbors within the subgraph. The authors proposed a greedy algorithm for discovering large quasi-cliques in massive graphs derived from telephone call records. While the work established the quasi-clique as a practical relaxation of the clique, the greedy approach does not guarantee the discovery of all maximal dense substructures and may miss valid solutions.

Komusiewicz and Hüffner [13] studied the parameterized complexity of finding dense subgraphs under several relaxed clique models, including  $s$ -plex and  $\gamma$ -quasi-clique. They showed that the problem is NP-hard for most density models and developed fixed-parameter tractable algorithms for small solution sizes. However, their algorithms are designed for the decision or single-solution variant rather than complete enumeration.

**Reverse-search enumeration framework.** Uno [6] proposed the first output-sensitive algorithm for enumerating all maximal pseudo-cliques using a reverse-search framework. The resulting *Pseudo-Clique Enumerator (PCE)* defines a deterministic parent function over the solution space, inducing a tree that can be traversed via depth-first search with polynomial delay per output. This framework avoids

the memory overhead of BFS-based approaches such as ODES [14], and it forms the structural foundation for all subsequent exact pseudo-clique enumerators, including the one presented in this work. However, the original PCE lacks structural pruning strategies, and its per-recursion overhead limits scalability on large sparse real-world graphs.

Long and Ester [14] proposed ODES, an output-sensitive algorithm based on a breadth-first enumeration strategy. While ODES achieves polynomial delay, it requires maintaining a frontier of candidate sets in memory, which can grow prohibitively large on dense or high-degree graphs. This memory limitation motivated the shift toward DFS-based reverse-search strategies adopted by PCE and FPCE.

**Pruning strategies for pseudo-clique enumeration.** Rahman *et al.* [7] proposed the *Fast Pseudo-Clique Enumerator (FPCE)*, which builds on Uno’s reverse-search framework and introduces three structural pruning techniques: (i) a *degeneracy-based order bound* that limits the maximum size of any pseudo-clique using the graph’s degeneracy  $\xi_G$ , enabling early termination of deep branches; (ii) a *Turán-theorem-based clique-existence filter* that determines a minimum clique size  $R$  that must exist within any feasible  $(\ell, \theta)$ -pseudo-clique, allowing branches lacking such a clique to be pruned; and (iii) a *core-aware edge bound* that uses the  $k$ -core structure to compute upper bounds on achievable edge counts, pruning partial sets that cannot reach the required density. FPCE achieves speedups of up to  $40\times$  over PCE on large sparse graphs. Nevertheless, FPCE relies on general-purpose data structures (e.g., standard adjacency lists and generic containers) whose per-recursion constant-factor overhead accumulates over billions of recursive calls.

Liu and Wong [15] developed the QUICK algorithm for mining quasi-cliques from large graphs using a depth-first branch-and-bound framework. QUICK integrates pruning based on vertex degrees and graph diameter and was among the first practical algorithms for quasi-clique discovery. However, QUICK targets a per-vertex minimum degree definition of quasi-cliques rather than the edge-density-based for-

mulation, and its pruning techniques are less effective on very sparse graphs where degeneracy and core structure provide stronger structural information.

**$k$ -Plex and alternative relaxed clique models.** Zhai *et al.* [16] focused on the enumeration of  $k$ -plexes, a relaxed clique model in which every vertex in a subgraph of size  $n$  must be adjacent to at least  $n - k$  other vertices. They introduced a *Connection Lower Bound (CLB)* technique that enables monotonic pruning during DFS traversal, and demonstrated scalability on graphs with over 800,000 vertices. While  $k$ -plexes and pseudo-cliques share the goal of relaxing clique strictness, the two models are structurally distinct:  $k$ -plex feasibility is hereditary (subsets of a  $k$ -plex are also  $k$ -plexes), whereas pseudo-clique feasibility is not, making pseudo-clique enumeration fundamentally harder to prune.

**Maximum clique and  $k$ -clique listing.** Chang *et al.* [17] introduced MC-BRB, a branch-reduce-and-bound framework for finding the maximum clique in large sparse graphs. MC-BRB combines graph reduction rules with branch-and-bound search and achieves strong performance on real-world networks. While maximum clique finding is a different problem from pseudo-clique enumeration (optimization vs. enumeration), the reduction and bounding techniques developed in MC-BRB have influenced pruning strategies in dense subgraph enumeration.

Wang *et al.* [18] proposed the *EBBkC* algorithm for exact  $k$ -clique listing using an edge-oriented branching strategy. EBBkC partitions the search space by edges rather than vertices, reducing redundant work and achieving state-of-the-art performance on large graphs. In our work, we leverage EBBkC as a streaming subroutine to enumerate the  $R$ -cliques required for Turán-based seeding, directly integrating its output into DensePCE’s search initialization.

Danisch *et al.* [11] developed efficient algorithms for listing  $k$ -cliques in large real-world graphs using core-based ordering and parallelism. Their work demonstrated that  $k$ -clique listing, while worst-case expensive, can be made practical on sparse net-



works through careful exploitation of graph degeneracy. This observation supports our decision to use exact  $k$ -clique listing (via EBBkC) as a practical seed-generation step rather than treating clique existence merely as a Boolean filter.

**Degeneracy and core decomposition.** Matula and Beck [8] introduced the  $k$ -core decomposition and the concept of graph degeneracy, which can be computed in linear time  $O(n + m)$  by iteratively removing minimum-degree vertices. Batagelj and Zaveršnik [9] refined this algorithm for practical use. The degeneracy ordering and core numbers derived from this decomposition serve as foundational building blocks for both the order-bound pruning and the core-aware edge bound used in FPCE and DensePCE.

### 2.1.1 Limitations of Existing Work

After a detailed examination of the literature, the following observations and limitations have been identified:

- (i) **Implementation overhead in exact enumerators.** Existing exact pseudo-clique enumerators, including PCE and FPCE, rely on general-purpose data structures such as adjacency lists implemented via standard library containers (`std::vector`, `std::set`). These structures incur pointer indirection, poor cache locality, and non-trivial allocation overhead during the billions of recursive add/remove operations that characterize large enumeration instances. No prior work has systematically investigated the impact of memory layout and data-structure design on reverse-search enumeration performance.
- (ii) **Passive use of the Turán property.** FPCE employs the Turán-derived clique-existence condition as a *filter*—checking whether a required  $R$ -clique can exist in a candidate neighborhood and pruning if not. However, this is a passive, Boolean test applied during search. No prior work has operationalized this property as an *active initialization strategy* by enumerating all  $R$ -cliques upfront and using them as seeds to bypass the expensive, high-branching-factor upper

levels of the recursion tree.

- (iii) **Limited integration of modern clique-listing subroutines.** State-of-the-art  $k$ -clique listers such as EBBkC [18] have demonstrated excellent practical performance, yet they have not been integrated into pseudo-clique enumeration pipelines. The potential for streaming exact clique seeds into a reverse-search enumerator remains unexplored.
- (iv) **Lack of fine-grained ablation studies.** Prior experimental evaluations typically compare full algorithm variants (PCE vs. FPCE) as monolithic systems. Detailed ablation studies that isolate the individual contributions of data-structure improvements, each pruning technique, and seeding strategies have not been conducted, making it difficult to attribute observed speedups to specific design choices.
- (v) **Incremental degree maintenance.** Existing enumerators recompute degree-related quantities (minimum internal degree, add-degree thresholds, maximality checks) by scanning vertex sets at each recursion node. The use of bucket-based data structures for  $O(1)$  amortized degree lookups and updates during reverse-search enumeration has not been explored in the pseudo-clique literature.

### 2.1.2 Motivation for This Work

The limitations identified above have motivated us to develop DensePCE, an optimized exact pseudo-clique enumerator that addresses these gaps through a combination of systems-level and algorithmic innovations. Specifically, we (i) replace general-purpose adjacency representations with a sorted Compressed Sparse Row (CSR) format to achieve cache-friendly memory access; (ii) introduce intrusive bucket-based degree lists for  $O(1)$  incremental state maintenance during recursion; (iii) integrate all three FPCE pruning techniques (order bound, core-aware edge bound, and Turán-based filtering) into this optimized engine; (iv) operationalize the Turán property as an active  $R$ -clique seeding strategy by streaming seeds from the EBBkC exact  $k$ -clique

lister; and (v) conduct ablation experiments that isolate the contribution of each component. Together, these contributions demonstrate that careful engineering of data structures and algorithmic initialization can yield substantial practical speedups over the state-of-the-art, while preserving the theoretical guarantees of exact reverse-search enumeration.

**Gap addressed by this work.** Existing exact pseudo-clique enumerators mainly reduce runtime by pruning the search space or by improving data structures. In contrast, our work studies a more specific source of overhead inside the reverse-search tree: repeated construction and validation of candidate pools across adjacent states. The first contribution, Candidate Uncle Inheritance, shows that under a sufficient gate the feasible candidate pool of a state can be inherited from the parent side instead of being rebuilt. The second contribution, the Outside-Gap theorem, identifies a structural case where the feasible pool is already the exact child set. These results complement FPCE’s pruning rules by targeting overhead that remains even after the search space itself has already been pruned.

# Chapter 3 Methodology

## 3.1 System Design

This section presents the overall system design of DensePCE and explains how the proposed methodology is organized at a high level. Since the objective of the work is to perform exact enumeration of maximal  $(\ell, \theta)$ -pseudo-cliques efficiently, the design focuses on structuring the search process, controlling recursion, and integrating optimization strategies without affecting correctness. The following subsections describe the methodological organization, the reverse-search framework, the seed-based acceleration strategy, and the complete workflow used by the system.

### 3.1.1 Methodological Overview

DensePCE is an optimized exact pseudo-clique enumerator designed to preserve the correctness of classical reverse-search enumeration while improving its practical runtime. The system targets the exact enumeration of all maximal  $(\ell, \theta)$ -pseudo-cliques in an undirected graph, where  $\ell$  denotes the minimum size of a valid solution and  $\theta$  denotes the minimum density threshold. The overall design combines three ideas: a cache-friendly graph layout, an incremental state-maintenance mechanism for recursion, and pruning strategies that reduce the number of search states explored.

The system is organized around a depth-first reverse-search framework. Instead of testing arbitrary subsets of vertices, the algorithm grows a current vertex set  $P \subseteq V$  one vertex at a time and recursively explores only those expansions that remain feasi-

ble under the pseudo-clique density constraint. This design ensures that the search is systematic, duplicate-free, and exact.

### 3.1.2 Reverse-Search Design

Each recursive state corresponds to a current partial solution  $P$ . Starting from an initial state, the algorithm attempts to generate child states of the form  $P \cup \{u\}$ , where  $u \notin P$ . The recursion continues as long as the expanded set remains feasible and no pruning rule proves that the current branch cannot lead to a valid maximal solution.

A reverse-search parent rule is used to ensure uniqueness of enumeration. Under this design, every feasible pseudo-clique has exactly one valid parent in the search tree. As a result, each maximal pseudo-clique is generated exactly once, and redundant traversals are avoided. This is a key design choice because pseudo-clique enumeration has a very large combinatorial search space, and even small amounts of duplication can significantly increase runtime.

The system follows a depth-first search strategy because it naturally supports rollback-based state updates. When a vertex is added to  $P$ , all relevant counters are updated incrementally. After the recursive call finishes, the same updates are reversed to restore the previous state. This makes the design memory-efficient and suitable for exact enumeration.

### 3.1.3 Seeded Search Strategy

The design supports two modes of search initialization. In the basic mode, the search starts from singleton vertex sets. In the accelerated mode, the search starts from pre-computed clique seeds. For a given  $(\ell, \theta)$ , the algorithm computes a clique size  $R$  such that every valid  $(\ell, \theta)$ -pseudo-clique must contain at least one  $R$ -clique. The search can then begin directly from such seeds instead of exploring the full upper portion of the recursion tree.

The value of  $R$  is computed as

$$R = \left\lceil \frac{1}{1 - \theta \cdot \frac{\ell-1}{\ell}} \right\rceil. \quad (1)$$

This design reduces unnecessary expansion near the root of the search tree and makes the system substantially faster on large sparse graphs.

### 3.1.4 Candidate-Pool Optimization Beyond the Baseline Design

The baseline DensePCE design already reduces the search space using the order bound, edge bound, and Turán-based filtering. However, after those pruning rules are applied, each recursive state still needs to identify the feasible outside vertices and then determine which of them are actual children in the reverse-search tree. In our investigation, this per-state overhead remained important on large sparse graphs. To address it, we added two theorem-backed refinements.

### 3.1.5 Candidate Uncle Inheritance

The original FPCE child-generation procedure performs two logically separate tasks at every recursive state  $P$ : it first identifies the outside vertices that are feasible with respect to the pseudo-clique density threshold, and then it filters those candidates using the reverse-search child conditions so that only the valid children of  $P$  are explored [6, 7]. In our investigation, we observed that the first task remains partially redundant across consecutive states on a root-to-leaf path. When  $P$  is obtained from its parent  $P'$ , a large fraction of the vertices that are feasible for  $P$  were already feasible for  $P'$ . This suggests that the feasible pool should be *inherited* instead of rebuilt from scratch.

Let  $P$  be a current pseudo-clique, let  $k = |P|$ , let  $v^*(P)$  denote the minimum-degree vertex of  $G[P]$  under the minimum-degree ordering, and let  $P' = P \setminus \{v^*(P)\}$ . We write

$$D_\psi(P) = \{u \notin P : d_P(u) \geq \psi(P)\},$$

where

$$\psi(P) = \left\lceil \theta \binom{|P|+1}{2} \right\rceil - |E[P]|.$$

Thus,  $D_\psi(P)$  is the set of all outside vertices that can extend  $P$  while preserving the density threshold.

**Motivating observation.** Suppose a vertex  $u$  is feasible for the parent state  $P'$ . When the single vertex  $v^*(P)$  is added back to obtain  $P$ , the degree of  $u$  into the current set can increase by at most one. Therefore, if the threshold increase from  $\psi(P')$  to  $\psi(P)$  is controlled, many vertices of  $D_\psi(P')$  remain feasible for  $P$ . This leads to the following inheritance rule.

**Theorem (Candidate Uncle Inheritance).** Let  $P$  be a current  $\theta$ -pseudo-clique and let  $P' = P \setminus \{v^*(P)\}$ . If

$$\theta|P| \geq \delta(P) + 1,$$

then

$$D_\psi(P) \subseteq D_\psi(P').$$

**Proof.** Take any  $u \in D_\psi(P)$ . By definition,  $d_P(u) \geq \psi(P)$ . Since  $P = P' \cup \{v^*(P)\}$ , we have

$$d_{P'}(u) \geq d_P(u) - 1.$$

Also,

$$\psi(P) - \psi(P') = \left\lceil \theta \binom{|P|+1}{2} \right\rceil - |E[P]| - \left( \left\lceil \theta \binom{|P|}{2} \right\rceil - |E[P']| \right).$$

Because  $|E[P]| = |E[P']| + \delta(P)$ , the threshold increment is bounded by

$$\psi(P) - \psi(P') \leq \lceil \theta|P| \rceil - \delta(P).$$

Under the assumption  $\theta|P| \geq \delta(P) + 1$ , we get  $\lceil \theta|P| \rceil - \delta(P) \geq 1$ , which yields

$$\psi(P') \leq \psi(P) - 1.$$

Therefore,

$$d_{P'}(u) \geq d_P(u) - 1 \geq \psi(P) - 1 \geq \psi(P').$$

Hence  $u \in D_\psi(P')$ , proving the claim.

**Interpretation.** This theorem does not yet say that inherited vertices are automatically children of  $P$ . Rather, it states that the parent-side feasible pool is a *superset* of the current feasible pool under the inheritance gate. Thus, instead of rebuilding  $D_\psi(P)$  from the whole outside boundary, one may start from  $D_\psi(P')$  and keep only those vertices that remain feasible for  $P$ . In other words, Candidate Uncle Inheritance removes a substantial part of the *candidate-pool construction cost* while preserving correctness.

**What remains expensive after inheritance.** Even after the feasible pool has been inherited, the algorithm still needs to determine which feasible candidates are actual *children* of  $P$  in the reverse-search tree. According to the child characterization of PCE, this requires checking not only pseudo-clique feasibility but also an ordering condition and an adjacency condition against the predecessors of a candidate [6]. In our experiments, this second phase became the main residual bottleneck of the inheritance path. The Outside-Gap theorem presented next was derived precisely to simplify this remaining cost.

### 3.1.6 Outside-Gap Theorem

Candidate Uncle Inheritance reduces the cost of obtaining the feasible candidate pool, but it does not by itself eliminate the need to validate whether each feasible vertex is an actual child of the current state  $P$ . In the PCE framework, this validation is governed by the child conditions of the reverse-search rule [6]. The most expensive remaining part is the predecessor-adjacency condition, which depends on the relative ordering between an outside vertex and the vertices already inside  $P$ .

To simplify this remaining step, we looked for a structural condition under which every feasible outside vertex automatically satisfies the child conditions. This led to



the notion of an *outside gap*.

Define

$$M(P) = \max_{u \notin P} d_P(u),$$

that is, the maximum number of neighbors inside  $P$  possessed by any outside vertex. Intuitively,  $M(P)$  measures how strongly the boundary of  $P$  connects back to the interior of  $P$ .

**Theorem (Outside Gap).** Let  $P$  be a  $\theta$ -pseudo-clique. If

$$M(P) < \delta(P),$$

then

$$\text{Children}(P) = D_\psi(P).$$

That is, under the outside-gap condition, an outside vertex is a child of  $P$  if and only if it is  $\psi$ -feasible for  $P$ .

**Proof.** We prove both set inclusions.

First, let  $u \in D_\psi(P)$ . By definition,

$$d_P(u) \geq \psi(P).$$

Since  $u \notin P$ , we also have

$$d_P(u) \leq M(P) < \delta(P).$$

Thus,

$$\psi(P) \leq d_P(u) < \delta(P).$$

Now consider the three child conditions of the PCE reverse-search rule.

(i) *Feasibility.* This holds directly because  $u \in D_\psi(P)$ .

(ii) *Lexicographic condition.* Since  $d_P(u) < \delta(P)$ , we immediately obtain

$$(d_P(u), u) < (\delta(P) + 1, v^*(P)).$$

Hence the ordering condition is satisfied.

(iii) *Predecessor-adjacency condition.* Every vertex  $x \in P$  has degree at least  $\delta(P)$  in  $G[P]$ , whereas  $u$  has degree strictly smaller than  $\delta(P)$  into  $P$ . Therefore no vertex of  $P$  can precede  $u$  in the minimum-degree ordering used by PCE. Consequently, the predecessor-adjacency requirement is vacuously satisfied.

Hence  $P \cup \{u\}$  is a valid child of  $P$ , and therefore

$$D_\psi(P) \subseteq \text{Children}(P).$$

Conversely, if  $u \in \text{Children}(P)$ , then by the child rule of PCE the feasibility condition must hold, i.e.,

$$d_P(u) \geq \psi(P).$$

So  $u \in D_\psi(P)$ , giving

$$\text{Children}(P) \subseteq D_\psi(P).$$

Combining the two inclusions yields

$$\text{Children}(P) = D_\psi(P).$$

This completes the proof.

**Interpretation.** The inequality  $M(P) < \delta(P)$  says that the strongest outside vertex is still weaker, with respect to  $P$ , than the weakest inside vertex of  $P$ . Under this strict separation, any outside vertex that passes the pseudo-clique feasibility threshold automatically lies early enough in the minimum-degree ordering that the additional child checks of PCE become unnecessary. Thus, the expensive child-validation stage

collapses to a single feasibility filter.

**Composition with Candidate Uncle Inheritance.** When both the inheritance condition and the outside-gap condition hold, the two results combine naturally. Candidate Uncle Inheritance states that the feasible pool of  $P$  can be inherited from the parent-side feasible pool. The Outside-Gap theorem states that this feasible pool is already the exact child set. Therefore, under the combined gate, the algorithm can reuse the inherited feasible pool and avoid the per-candidate child-validation step entirely. This does not change the search tree or the output; it only reduces the overhead of generating the next recursive states.

### 3.1.7 System-Level Workflow

At a high level, the full methodology follows the workflow below:

1. Load the input graph and convert it into a sorted CSR representation.
2. Compute graph degeneracy and core numbers for structural pruning.
3. Determine whether the target parameters admit any feasible pseudo-clique using the order bound.
4. Either start from singleton vertices or generate valid  $R$ -clique seeds.
5. During reverse-search enumeration, maintain the current feasible pool incrementally and apply Candidate Uncle Inheritance whenever the inheritance gate holds.
6. If the Outside-Gap condition also holds at the current state, use the feasible pool directly as the exact child set; otherwise, apply the usual child-validation rule.
7. Output all maximal  $(\ell, \theta)$ -pseudo-cliques along with execution statistics.

This extended design keeps the algorithm exact while making the search process more scalable in practice. The key point is that the new refinements do not alter the

search tree logically; rather, they reduce the amount of work required to generate that same search tree.

## **3.2 Hardware and/or Software Components**

This section describes the major components that support the proposed methodology. As DensePCE is a software-based graph mining system, its components are primarily algorithmic and data-structural rather than hardware-oriented. Each component has a specific role in ensuring efficient execution, including graph representation, recursive state maintenance, candidate management, structural preprocessing, seed generation, and result reporting. The following subsections explain these components in detail.

### **3.2.1 Software Components of the Methodology**

The methodology is implemented entirely as a software-driven graph mining system. Since the project focuses on exact pseudo-clique enumeration rather than hardware prototyping, the key components are algorithmic and data-structural in nature. The main software components are described below.

### **3.2.2 Graph Storage Component**

The input graph is stored using the Compressed Sparse Row (CSR) format. In this representation, adjacency lists are placed in contiguous memory and kept in sorted order. This design provides two advantages. First, neighborhood traversal becomes efficient because adjacent vertices can be accessed sequentially. Second, adjacency checking can be performed using binary search on a sorted list, which reduces lookup overhead during recursion.

CSR is especially suitable for this problem because the algorithm repeatedly performs neighborhood-based updates and connectivity checks while constructing candidate pseudo-cliques. Compared with naive adjacency structures, CSR offers better cache locality and lower overhead on large sparse graphs.

### 3.2.3 State-Maintenance Component

The algorithm maintains a dynamic recursive state for the current solution set  $P$ . The following quantities are tracked throughout the search:

- a membership indicator  $\text{inP}[v]$  that tells whether vertex  $v$  is currently inside  $P$ ,
- a degree-to- $P$  counter for every vertex,
- the current size  $|P|$ , and
- the number of induced edges  $|E[P]|$ .

For a vertex  $x \in V$ , the degree-to- $P$  counter is defined as

$$\text{deg}_{\rightarrow P}(x) = |\{u \in P : (u, x) \in E\}|. \quad (2)$$

When  $x \in P$ , this value gives the internal degree of  $x$  inside the current induced subgraph. When  $x \notin P$ , the same value indicates how strongly  $x$  is connected to the current state and whether adding it can preserve pseudo-clique feasibility.

### 3.2.4 Intrusive Bucket Component

To avoid repeatedly scanning all vertices, the implementation uses intrusive degree-indexed buckets. Vertices are grouped according to their current degree-to- $P$  values. Two separate bucket families are maintained:

- $P$ -buckets for vertices already inside  $P$ , and
- non- $P$  buckets for vertices outside  $P$ .

Each bucket is implemented as an intrusive doubly linked list. When a vertex is added to or removed from  $P$ , the degree-to- $P$  values of its neighbors change, and those vertices are moved between buckets in constant unlink/insert time per move. This component is essential for quickly identifying the minimum internal degree of the current state and for scanning only promising candidate vertices.

### 3.2.5 Structural Analysis Component

Another important component is the graph structural preprocessing module. Before enumeration begins, the implementation computes core numbers for all vertices and the graph degeneracy. These values support two major optimizations: the order bound and the clique-seeding restriction to a smaller core-induced subgraph.

The degeneracy value, denoted by  $\xi_G$ , is used to estimate an upper bound on the maximum possible size of a pseudo-clique under the chosen density threshold. Core numbers are also used in the edge-bound pruning step and in restricting the seed-generation phase to the  $(R - 1)$ -core.

### 3.2.6 Seed Generation Component

The methodology includes a seed-generation module based on exact clique listing. Once the required clique size  $R$  is determined, the system enumerates all valid  $R$ -cliques from the restricted core subgraph and uses them as search seeds. This component acts as an accelerator because it bypasses many shallow recursion states that do not contribute meaningfully to final solutions.

### 3.2.7 Output and Statistics Component

The final software component is the reporting layer. In addition to listing maximal pseudo-cliques, the system records execution statistics such as the total number of recursive iterations, the number of pruned subtrees, and solution counts grouped by pseudo-clique size. These outputs are useful for correctness verification, performance comparison, and ablation analysis.

## 3.3 Hardware and/or Software Implementation

This section explains how the proposed methodology is realized in the actual implementation of DensePCE. While the previous sections describe the design principles

and supporting components, this section focuses on the operational logic used during execution, including feasibility checking, maximality verification, child generation, pruning, seed initialization, and rollback-based recursion. The following subsections present the main implementation mechanisms that make the exact enumeration process both correct and efficient.

### 3.3.1 Feasibility Test During Expansion

Let the current recursive state be  $P$  with  $|P| = p$  and  $|E[P]| = e$ . If a vertex  $u \notin P$  is added, the new state  $P \cup \{u\}$  is feasible only if it satisfies the pseudo-clique density constraint. This condition is expressed as

$$e + \deg_{\rightarrow P}(u) \geq \theta \binom{p+1}{2}. \quad (3)$$

To simplify repeated checking, the implementation computes

$$\theta_P = \theta \binom{|P|+1}{2} - |E[P]|, \quad (4)$$

and then requires

$$\deg_{\rightarrow P}(u) \geq \lceil \theta_P \rceil. \quad (5)$$

Only vertices meeting this threshold are considered for expansion.

### 3.3.2 Maximality Check

Whenever the current set satisfies  $|P| \geq \ell$ , the algorithm checks whether it is maximal. A set is maximal if no vertex outside  $P$  can be added while maintaining feasibility. Using the non- $P$  degree buckets, this check becomes efficient: if there exists no outside vertex whose degree-to- $P$  meets the required threshold, then the current set is output as a maximal pseudo-clique.

### 3.3.3 Uniqueness-Aware Child Generation

Child generation is not performed by blindly testing all outside vertices. Instead, the implementation partitions candidates according to their degree-to- $P$  values and applies ordering rules derived from reverse search. These rules ensure that each feasible pseudo-clique is reached from exactly one parent state.

The implementation uses a pivot vertex  $v^*$  taken from the minimum-degree portion of the current state. Depending on a candidate's relation to the current minimum internal degree, the candidate may be accepted directly or may need to satisfy additional adjacency and ordering conditions. This implementation detail is critical for maintaining exactness without duplicate outputs.

### 3.3.4 Order-Bound Implementation

Before starting enumeration, the algorithm computes a degeneracy-based upper bound on the possible order of any pseudo-clique under the selected threshold. Let  $\xi_G$  denote the graph degeneracy. Two upper bounds are computed:

$$\mu_1 = \left\lfloor \frac{2\xi_G}{\theta} \right\rfloor, \quad (6)$$

$$\mu_2 = \begin{cases} \left\lfloor \frac{1}{1 - \frac{\xi_G}{(\xi_G+1)\theta}} \right\rfloor, & \text{if } \theta > \frac{\xi_G}{\xi_G + 1}, \\ \infty, & \text{otherwise.} \end{cases} \quad (7)$$

The algorithm then sets

$$\mu = \min(\mu_1, \mu_2). \quad (8)$$

If  $\ell > \mu$ , no valid  $(\ell, \theta)$ -pseudo-clique can exist, so the program terminates immediately. Otherwise, recursion is not allowed to expand beyond size  $\mu$ . This reduces unnecessary exploration in cases where the graph structure itself rules out larger dense subgraphs.



### 3.3.5 Turán-Based Clique Seeding Implementation

To accelerate the upper levels of the search tree, the implementation uses Turán-based clique seeding. First, it computes the required clique size

$$R = \left\lceil \frac{1}{1 - \theta \cdot \frac{\ell-1}{\ell}} \right\rceil. \quad (9)$$

Then the algorithm induces the  $(R - 1)$ -core of the graph and enumerates all  $R$ -cliques from that restricted instance. Each discovered clique is inserted directly into the recursive state as an initial seed, after which reverse search continues normally. This avoids exploring many weak partial states that would never grow into valid solutions.

### 3.3.6 Core-Aware Edge-Bound Pruning

A further pruning rule is applied when the current set is still smaller than the required solution size. For the target parameters, the implementation precomputes the minimum number of edges required for any feasible solution of size  $\ell$ :

$$L = \left\lceil \theta \binom{\ell}{2} \right\rceil. \quad (10)$$

If the current state is  $P$ , let  $t = \ell - |P|$ , let  $\delta(P)$  be the minimum internal degree in  $P$ , and let  $c(P)$  be the minimum core number among vertices in  $P$ . Then the implementation defines

$$\eta(P) = \min\{c(P), \delta(P) + t\}, \quad g(P) = c(P) - \delta(P) - 1. \quad (11)$$

Using these quantities, the maximum achievable edge count after extending  $P$  to size  $\ell$  is bounded. If the required value  $L$  exceeds that upper bound, the entire recursion subtree rooted at  $P$  is pruned because no extension can possibly satisfy the density requirement.

### **3.3.7 Deterministic and Reproducible Execution**

To keep the implementation reproducible, candidate vertices are expanded in deterministic order. Seed cliques are also sorted consistently before recursion begins. This does not change the output set, but it ensures that repeated runs generate the same traversal order and the same output ordering.

### **3.3.8 Rollback-Based Execution Flow**

The full implementation follows an add–recurse–rollback pattern. When a feasible candidate is selected, the algorithm inserts it into the current state, updates all counters and bucket memberships incrementally, and then performs the recursive call. After returning from recursion, the same updates are reversed exactly. This approach avoids reconstructing the state from scratch at each step and is one of the main reasons for the efficiency of DensePCE.

### **3.3.9 Implementation Outcome**

Overall, the implementation remains an exact enumerator, meaning that it preserves the output semantics of reverse-search pseudo-clique enumeration. The improvements come from more efficient state maintenance and from pruning rules that reduce the number of branches explored, not from approximation or heuristic omission of solutions.

---

**Algorithm 1** FPCE\*: Enumerating maximal  $(\ell, \theta)$ -pseudo-cliques
 

---

**Require:** Graph  $G = (V, E)$ , minimum order  $\ell$ , minimum density  $\theta$

**Ensure:** All maximal  $(\ell, \theta)$ -pseudo-cliques in  $G$

```

1: Load  $G$  into sorted CSR; compute core numbers and degeneracy  $\xi_G$ 
2: Compute order bound  $\mu(\theta, \xi_G)$ 
3: if  $\ell > \mu$  then
4:   return
5: end if
6:  $R \leftarrow \left\lceil \frac{1}{1 - \theta \cdot \frac{\ell-1}{\ell}} \right\rceil$ 
7: if  $R \geq 3$  and Turán seeding enabled then
8:   Induce  $(R-1)$ -core subgraph  $H$  and stream all  $R$ -cliques via EBBkC
9:   for all seed clique  $C$  streamed from EBBkC do
10:    Build  $P \leftarrow C$  into the incremental state
11:    Choose pivot  $v^*$  from the min-degree bucket of  $P$ 
12:    RECUR( $P, v^*, \emptyset$ )
13:    Roll back  $P$ 
14:   end for
15: else
16:   for  $v \in V$  do
17:    Build  $P \leftarrow \{v\}$ 
18:    RECUR( $P, v, \emptyset$ )
19:    Roll back  $P$ 
20:   end for
21: end if

22: procedure RECUR( $P, v^*, D_\psi^{\text{par}}$ )
23:   if edge-bound enabled and  $1 \leq |P| < \ell$  and EDGEBOUNDPRUNE( $P$ ) then
24:     return
25:   end if
26:   if  $|P| \geq \ell$  and ISMAXIMAL( $P$ ) then
27:     output  $P$ 
28:   end if
29:   if order bound enabled and  $|P| = \mu$  then
30:     return
31:   end if
32:    $\psi(P) \leftarrow \left\lceil \theta \binom{|P|+1}{2} \right\rceil - |E[P]|$ 
33:   Compute  $\delta(P)$  from the degree buckets
34:   if CUI enabled and INHERITANCEGATE( $P$ ) holds then
35:      $D_\psi(P) \leftarrow \text{FILTERFEASIBLE}(D_\psi^{\text{par}}, \psi(P))$ 
36:   else
37:      $D_\psi(P) \leftarrow \text{GENERATEFROMBUCKETS}(P, \psi(P), \delta(P))$ 
38:   end if
39:   if Outside-Gap enabled and  $\max_{u \notin P} d_P(u) < \delta(P)$  then
40:      $\mathcal{C}(P) \leftarrow D_\psi(P)$ 
41:   else
42:      $\mathcal{C}(P) \leftarrow \text{FILTERCHILDREN}(P, D_\psi(P))$ 
43:   end if
44:   for all  $u \in \mathcal{C}(P)$  in deterministic order do
45:     Add  $u$  to  $P$  using incremental updates
46:     RECUR( $P \cup \{u\}, u, D_\psi(P)$ )
47:     Remove  $u$  from  $P$  using rollback updates
48:   end for
49: end procedure

```

---

# Chapter 4 Investigation/Experiment, Result, Analysis and Discussion

## 4.1 Investigation and Experimental Setup

This section presents the investigation and experimental procedure used to evaluate DensePCE. Since the goal of the study is to compare the practical behavior of exact pseudo-clique enumeration methods, the investigation focuses on dataset preparation, preprocessing, execution environment, evaluation criteria, and output verification. The following subsections describe the benchmark graphs, the preprocessing pipeline, the hardware and software environment, the metrics used for comparison, and the correctness checks applied before analyzing runtime behavior.

### 4.1.1 Benchmark Graphs and Preprocessing

The evaluation was conducted on ten real-world undirected graphs that are commonly used in dense subgraph mining and clique-related research. These datasets vary substantially in size, density, and degeneracy, making them suitable for testing the scalability and robustness of exact pseudo-clique enumeration methods. Table 4.1 summarizes the graph statistics used in this study.

Table 4.1: Properties of the real graph datasets used in the experiments. Here,  $\text{den}(G)$  denotes graph density and  $D$  denotes graph degeneracy.

| ID | Graph Name        | V         | E          | Density               | D  |
|----|-------------------|-----------|------------|-----------------------|----|
| 1  | bio-grid-human    | 9,436     | 31,181     | $7.00 \times 10^{-4}$ | 12 |
| 2  | scc-retweet-crawl | 17,151    | 24,014     | $1.63 \times 10^{-4}$ | 19 |
| 3  | soc-gplus         | 23,628    | 39,193     | $1.40 \times 10^{-4}$ | 12 |
| 4  | tech-internet-as  | 40,164    | 85,122     | $1.05 \times 10^{-4}$ | 23 |
| 5  | PROTEINS-full     | 43,466    | 81,043     | $8.57 \times 10^{-5}$ | 4  |
| 6  | soc-douban        | 154,908   | 327,161    | $2.72 \times 10^{-5}$ | 15 |
| 7  | citation          | 268,495   | 1,156,646  | $3.20 \times 10^{-5}$ | 15 |
| 8  | delaunay_n23      | 8,388,608 | 25,165,783 | $7.15 \times 10^{-7}$ | 4  |
| 9  | com-amazon        | 334,863   | 925,871    | $1.65 \times 10^{-5}$ | 6  |
| 10 | TWITTER           | 580,768   | 717,557    | $4.25 \times 10^{-6}$ | 9  |

Before running the algorithms, each graph was preprocessed once using a Python-based conversion pipeline. First, the raw edge-list file was cleaned by removing self-loops and repeated edges, and the vertex identifiers were normalized into a contiguous range. Next, the cleaned graph was converted into binary arrays suitable for fast CSR-style loading and efficient seed enumeration. This preprocessing stage ensured that both FPCE and DensePCE were executed on the same normalized graph representation.

All executions in the reported comparison were performed with the same pseudo-clique parameters, namely minimum order  $\ell = 10$  and density threshold  $\theta = 0.9$ . Using a fixed parameter configuration across all datasets allowed the runtime differences to be attributed to implementation behavior and pruning effectiveness rather than to changes in the problem definition.

### 4.1.2 Experimental Setup

The experiments were performed on a workstation configured to support large-scale graph mining tasks. The hardware and operating environment used during evaluation are listed below:

- **CPU:** AMD Ryzen 9 5900X, 3.7 GHz, 12 cores and 24 threads
- **RAM:** 64 GB
- **Operating System:** Ubuntu 20.04 LTS, 64-bit

This setup provided enough memory and processing resources to evaluate exact enumeration on both medium-scale and large-scale sparse graphs. The same machine was used for all runs to keep the comparison consistent.

### 4.1.3 Evaluation Metrics

The primary metric used in this chapter is **CPU time**. Since FPCE and DensePCE both perform exact enumeration and produce the same type of output, runtime is the most direct measure for comparing practical efficiency. For every graph and every algorithmic configuration, CPU time was recorded and compared.

In addition to raw runtime, the study also reports **speedup**. For a graph instance  $i$ , if  $T_A(i)$  and  $T_B(i)$  denote the measured CPU time of methods  $A$  and  $B$ , respectively, then the speedup of  $B$  over  $A$  is defined as

$$\text{Speedup}_{A \rightarrow B}(i) = \frac{T_A(i)}{T_B(i)}. \quad (1)$$

A value greater than 1 indicates that method  $B$  is faster than method  $A$  for that graph, while a value less than 1 indicates the opposite.

### 4.1.4 Correctness Verification

Before interpreting the runtime results, the outputs of FPCE and DensePCE were verified for consistency. For every tested graph, the enumerated maximal  $(\ell, \theta)$ -pseudo-cliques produced by DensePCE were checked against those produced by FPCE. The verification confirmed that both methods enumerate identical solution sets for the tested instances. Therefore, the performance comparisons presented in this chapter are based on equivalent outputs and reflect implementation-level differences rather

than changes in correctness.

## 4.2 Results

This section presents the measured runtime results for DensePCE and FPCE under two settings: first without Turán-based  $R$ -clique seeding, and then with the complete optimization pipeline enabled. Since both algorithms enumerate the same maximal pseudo-cliques, the reported numbers highlight computational efficiency rather than output differences. The following subsections summarize the observed runtime behavior and present the tabulated and graphical comparisons.

### 4.2.1 Runtime Comparison Without Turán-Based Seeding

Table 4.2 compares FPCE and DensePCE when Turán-based seeding is disabled for both methods. This setting isolates the effect of the underlying implementation strategy and data-structure design.

Table 4.2: Comparison of CPU time between FPCE and DensePCE without Turán-based seeding. The speedup is computed as  $T_{\text{FPCE}} / T_{\text{DensePCE}}$ .

| Graph Name        | FPCE Without Turán (s) | DensePCE Without Turán (s) | Speedup |
|-------------------|------------------------|----------------------------|---------|
| Protein           | 0.03                   | < 0.01                     | > 3.0   |
| gplus             | 0.77                   | 0.57                       | 1.351   |
| bio-grid-human    | 0.42                   | 0.40                       | 1.050   |
| soc-douban        | 0.86                   | 10.13                      | 0.085   |
| TWITTER           | 1.26                   | 32.30                      | 0.039   |
| com-amazon        | 1.51                   | 94.60                      | 0.016   |
| citation          | 10.51                  | 370.22                     | 0.028   |
| delaunay_n23      | 10.51                  | 3.53                       | 2.977   |
| scc-retweet-crawl | 520.82                 | 583.13                     | 0.893   |
| tech              | 1275.38                | 1100.66                    | 1.159   |

The results in this setting show that DensePCE without Turán seeding is not uniformly better on every graph. It achieves runtime improvements on graphs such as Protein, gplus, bio-grid-human, delaunay\_n23, and tech, but FPCE remains faster on several other instances, particularly soc-douban, TWITTER, com-amazon, citation,

and scc-retweet-crawl. This indicates that data-structure refinement alone does not guarantee a consistent advantage across all graph structures.

#### 4.2.2 Runtime Comparison With Full Optimization

Table 4.3 reports the runtime comparison when the full optimization pipeline is enabled, including Turán-based  $R$ -clique seeding and the EBBkC-assisted seed generation used by DensePCE.

Table 4.3: Comparison of CPU time between FPCE and DensePCE with the complete optimization pipeline enabled. The speedup is computed as  $T_{\text{FPCE}} / T_{\text{DensePCE}}$ .

| Graph Name        | FPCE CPU Time (s) | DensePCE CPU Time (s) | Speedup |
|-------------------|-------------------|-----------------------|---------|
| Protein           | 0.04              | < 0.01                | > 4.0   |
| gplus             | 0.54              | 0.14                  | 3.857   |
| bio-grid-human    | 0.41              | 0.30                  | 1.367   |
| soc-douban        | 0.77              | 0.36                  | 2.139   |
| TWITTER           | 1.23              | 0.50                  | 2.460   |
| com-amazon        | 1.38              | 0.72                  | 1.917   |
| citation          | 10.20             | 2.75                  | 3.709   |
| delaunay_n23      | 11.79             | 3.78                  | 3.119   |
| scc-retweet-crawl | 535.54            | 12.28                 | 43.611  |
| tech              | 1313.27           | 718.02                | 1.829   |

In contrast to the previous setting, DensePCE with the full optimization pipeline outperforms FPCE on every tested graph. The improvement is modest on some datasets, such as bio-grid-human and com-amazon, but becomes very large on graphs such as citation, gplus, delaunay\_n23, and especially scc-retweet-crawl. These results indicate that the full combination of structural pruning, efficient state maintenance, and accelerated seed generation is necessary to obtain consistent runtime gains.

#### 4.2.3 Ablation of Candidate Uncle Inheritance and the Outside-Gap Theorem

Beyond the full DensePCE-versus-FPCE comparison, we further studied the two theorem-backed refinements introduced in this work. The first comparison, mode



4 versus mode 5, isolates the effect of Candidate Uncle Inheritance. The second comparison, mode 5 versus mode 6, isolates the marginal effect of the Outside-Gap theorem on top of inheritance. In both comparisons, the graph-dependent counts remain unchanged, which indicates that the new rules preserve correctness and do not alter the logical search tree.

#### 4.2.4 Results for Candidate Uncle Inheritance

To evaluate the effect of Candidate Uncle Inheritance independently, we compared mode 4 (baseline DensePCE with order bound, edge bound, and Turán filtering) against mode 5 (the same configuration plus Candidate Uncle Inheritance). The graph-dependent counts remained unchanged, which confirms that the inheritance rule does not alter the search space or the set of maximal pseudo-cliques reported. The observed differences were therefore attributable only to lower overhead in candidate-pool construction.

Table 4.4: Comparison between the baseline DensePCE configuration and DensePCE with Candidate Uncle Inheritance (CUI). Since the output-dependent graph statistics remain unchanged across the two methods, only runtime, peak memory, and CUI-specific savings are reported.

| Dataset           | Real time |         | Peak memory (MB) |        | Uncle-inheritance savings |
|-------------------|-----------|---------|------------------|--------|---------------------------|
|                   | Baseline  | CUI     | Baseline         | CUI    |                           |
| TWITTER           | 0:00.70   | 0:00.53 | 224.67           | 225.07 | 844                       |
| bio-grid-human    | 0:00.29   | 0:00.24 | 83.59            | 83.75  | 152,746                   |
| citation          | 0:02.73   | 0:02.60 | 401.69           | 402.01 | 1,127,645                 |
| com-amazon        | 0:01.03   | 0:00.86 | 295.83           | 296.15 | 0                         |
| delaunay_n23      | 0:05.30   | 0:05.26 | 355.96           | 355.97 | –                         |
| gplus             | 0:00.15   | 0:00.09 | 80.93            | 81.40  | 4,089                     |
| protein           | 0:00.06   | 0:00.02 | 5.31             | 5.45   | –                         |
| scc-retweet-crawl | 5:23.93   | 5:09.49 | 86.87            | 87.18  | 383,897,455               |
| soc-douban        | 0:00.38   | 0:00.28 | 97.11            | 97.10  | 95,408                    |
| tech              | 7:54.95   | 7:56.97 | 90.31            | 90.62  | 172,604,111               |

The comparison shows that Candidate Uncle Inheritance yields modest but consistent gains on several graphs, with the most meaningful absolute improvements appearing on the largest sparse datasets. The result is significant from a systems perspective because the optimization is exact and uses already available information from

the parent state. At the same time, the gains are not dramatic, which is expected: the theorem reduces only the candidate-pool construction cost, not the search-tree size itself.

#### 4.2.5 Results for the Outside-Gap Theorem

We then compared mode 5 and mode 6, where mode 6 augments Candidate Uncle Inheritance with the Outside-Gap fast path. Again, the graph-dependent counts remained unchanged across the two modes. Therefore, the runtime differences isolate the marginal effect of the Outside-Gap theorem on top of inheritance.

Table 4.5: Comparison between DensePCE with Candidate Uncle Inheritance (CUI) and DensePCE with CUI plus the Outside-Gap theorem. As the search space and graph-dependent counts remain unchanged, only runtime, peak memory, and Uncle saves are reported.

| Dataset           | Real time |                 | Peak memory (MB) |                 | Uncle saves |
|-------------------|-----------|-----------------|------------------|-----------------|-------------|
|                   | CUI       | CUI+Outside-Gap | CUI              | CUI+Outside-Gap |             |
| TWITTER           | 0:00.63   | 0:00.53         | 225.07           | 224.96          | 844         |
| bio-grid-human    | 0:00.23   | 0:00.23         | 83.90            | 83.75           | 152,746     |
| citation          | 0:03.14   | 0:02.94         | 401.95           | 401.69          | 1,127,645   |
| com-amazon        | 0:01.07   | 0:00.92         | 296.04           | 295.94          | 0           |
| delaunay_n23      | 0:05.74   | 0:05.09         | 355.96           | 355.80          | –           |
| gplus             | 0:00.11   | 0:00.10         | 81.09            | 81.25           | 4,089       |
| protein           | 0:00.02   | 0:00.02         | 5.45             | 5.45            | –           |
| scc-retweet-crawl | 5:09.49   | 4:54.31         | 87.18            | 87.18           | 383,897,455 |
| soc-douban        | 0:00.31   | 0:00.28         | 97.01            | 97.11           | 95,408      |
| tech              | 7:55.41   | 7:40.41         | 90.62            | 90.62           | 172,604,111 |

The mode-5 versus mode-6 comparison establishes three important findings. First, the theorem-backed fast path is correctness-preserving, since the maximal pseudo-clique counts and iteration counts remain unchanged. Second, the memory footprint is essentially unaffected, which matches the fact that the outside-gap condition is checked using already maintained degree information. Third, the runtime is never worse in the reported experiments and is measurably better on most datasets, including non-trivial absolute savings on scc-retweet-crawl and tech. This confirms that the Outside-Gap theorem functions as an *overhead-reduction* technique rather than as a new pruning rule: it does not reduce the number of visited states, but it makes some

child-generation steps cheaper.

#### 4.2.6 Discussion of the Combined Effect

Taken together, the results support the following interpretation. Candidate Uncle Inheritance removes redundant work in reconstructing the feasible candidate pool, whereas the Outside-Gap theorem removes part of the remaining cost of validating those candidates as children. The two techniques are therefore complementary. Candidate Uncle Inheritance addresses *where the candidate pool comes from*; Outside Gap addresses *when that pool can be used directly as the exact child set*. Their combination produces a cleaner and more efficient execution path without changing the logical behavior of the enumerator.

#### 4.2.7 Graphical Comparison

Figure( 4.1, 4.2)provides a visual summary of the CPU time comparisons for both configurations. The left subfigure compares the fully optimized versions, while the right subfigure compares FPCE and DensePCE without Turán-based seeding.

Figures 4.3 and 4.4 present the runtime comparison of FPCE and DensePCE on synthetically generated scale-free and small-world graphs, respectively, with the graph density parameter  $m$  varied from 5 to 40. Both plots use a logarithmic scale on the  $y$ -axis to accommodate the exponential growth in computation time as graph density increases.

On scale-free graphs (Figure 4.3), DensePCE consistently runs below FPCE across all tested values of  $m$ . The separation between the two curves is particularly pronounced in the mid-range densities ( $m = 10$ – $25$ ), where DensePCE exploits the hub-dominated structure of scale-free networks to prune the search space more aggressively. At higher densities ( $m \geq 35$ ), both algorithms enter a computationally expensive regime—FPCE exceeds 1,750 seconds at  $m = 40$ , while DensePCE completes in approximately 843 seconds—yet DensePCE maintains a consistent

advantage throughout.

On small-world graphs (Figure 4.4), a similar trend holds, though the absolute runtimes are higher for both algorithms at large  $m$ . FPCE reaches nearly 8,950 seconds at  $m = 40$ , whereas DensePCE requires approximately 5,319 seconds. The tighter gap between the two curves, compared to the scale-free case, reflects the more uniform clustering structure of small-world graphs, which provides fewer opportunities for the dense-subgraph pruning heuristics to eliminate large portions of the search tree.

**Speedup Analysis.** Figures 4.5 and 4.6 quantify the speedup that DensePCE achieves over FPCE, computed as the ratio of FPCE runtime to DensePCE runtime at each value of  $m$ .

On scale-free graphs (Figure 4.5), DensePCE achieves a peak speedup of approximately  $8.5\times$  at  $m = 15$  and maintains an average speedup of  $4.4\times$  across all tested densities. The speedup is highest in the mid-density range and gradually decreases at very high densities ( $m \geq 35$ ), where the sheer volume of pseudo-cliques limits the relative benefit of pruning. Nonetheless, even at  $m = 40$ , DensePCE remains approximately  $2.1\times$  faster than FPCE.

On small-world graphs (Figure 4.6), the speedup is more moderate but consistent, averaging  $1.85\times$  across all values of  $m$ . The peak speedup of  $3.0\times$  occurs at the sparsest configuration ( $m = 5$ ), after which the ratio stabilizes in the range of  $1.3\times$ – $1.9\times$ . This plateau behavior suggests that the implementation-level optimizations in DensePCE—such as the CSR-based adjacency representation and intrusive bucket lists—provide a steady constant-factor improvement that is largely independent of graph density in the small-world regime.

Taken together, these results demonstrate that DensePCE delivers meaningful performance gains across both graph topologies, with the benefit being most pronounced on scale-free networks where structural heterogeneity amplifies the effectiveness of dense-subgraph pruning strategies.

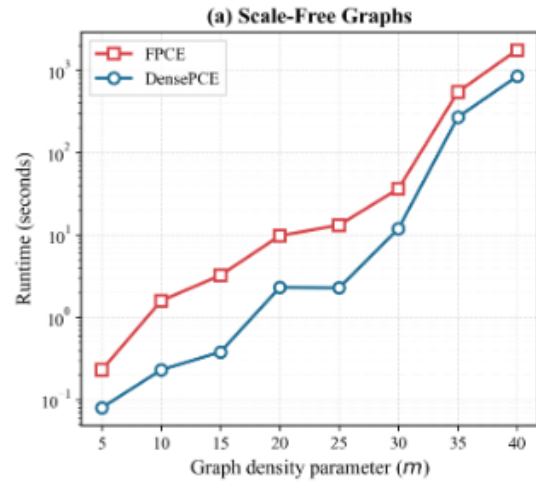


Figure 4.3: FPCE versus DensePCE on scale free graphs.

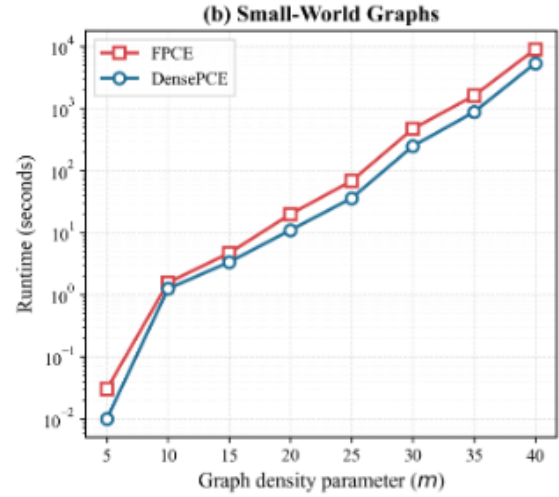


Figure 4.4: FPCE versus DensePCE on small world graphs.

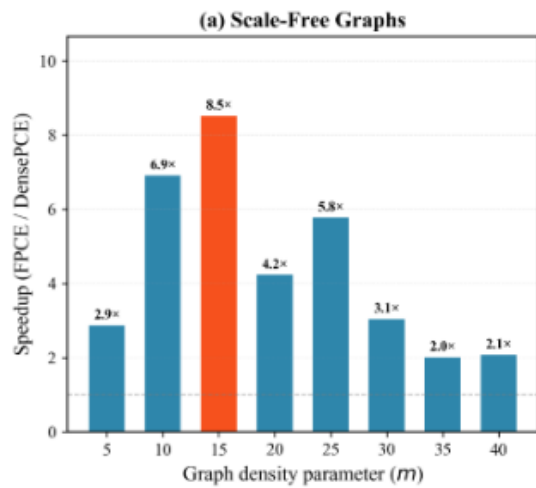


Figure 4.5: DensePCE speed up over FPCE on scale free graphs.

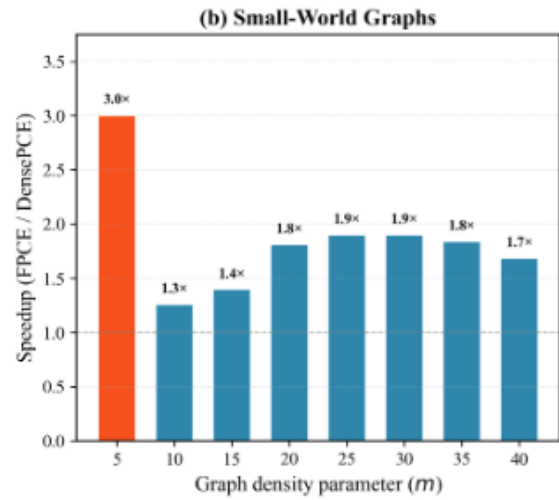


Figure 4.6: DensePCE speed up over FPCE on small world graphs.

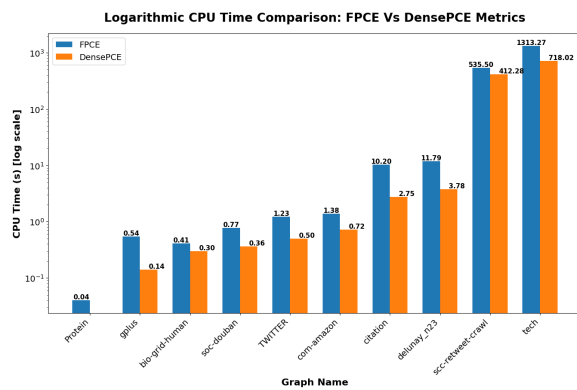


Figure 4.1: FPCE versus DensePCE with full optimization.

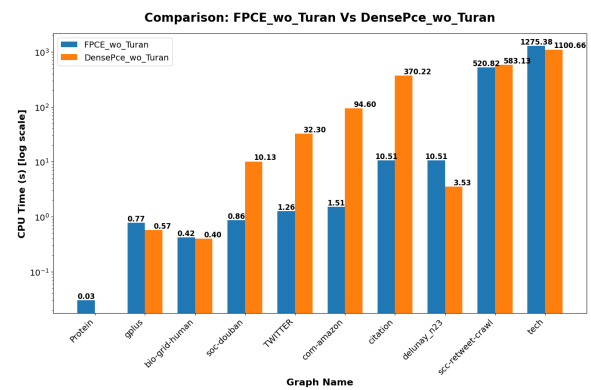


Figure 4.2: FPCE versus DensePCE without Turán-based seeding.

The graphical comparison supports the tabulated findings. The gap between FPCE and DensePCE becomes systematically favorable to DensePCE when the full optimization pipeline is enabled, whereas the no-seeding comparison shows that performance depends more strongly on dataset structure when only implementation-level changes are applied.

## 4.3 Analysis and Discussion

This section analyzes the experimental observations and discusses what the results imply about the proposed DensePCE framework. The goal is not only to report that one method is faster than another, but also to understand which design decisions contribute most to the observed behavior. The following subsections interpret the effect of the optimizations, summarize the ablation evidence, and discuss the practical significance of the findings.

### 4.3.1 Interpretation of the Runtime Trends

The experiments show that the runtime of both FPCE and DensePCE increases with graph size and with the difficulty of the exact enumeration task. This is expected because pseudo-clique enumeration remains exponential in the worst case, and practical performance depends heavily on how effectively the recursion tree can be reduced.

The no-seeding comparison reveals an important point: DensePCE does not outperform FPCE on every graph when only the underlying implementation is changed. This suggests that efficient data structures alone are beneficial but not always sufficient. The effect of CSR storage and intrusive degree buckets depends on the structure of the graph, the number of feasible candidate expansions, and the extent to which the recursion repeatedly reuses incremental state information.

The full-optimization comparison, however, shows a clear and consistent advantage for DensePCE across all tested graphs. This indicates that the best runtime behavior emerges from the combined action of multiple ideas rather than from a single

isolated improvement. In particular, DensePCE benefits when efficient implementation and search-space reduction techniques reinforce one another.

### 4.3.2 Ablation-Based Discussion

The ablation evidence helps separate the influence of different design choices. First, the comparison without Turán-based seeding evaluates the effect of implementation and data-structure improvements in relative isolation. In that setting, DensePCE is faster on several graphs, but the advantage is not universal. This confirms that the new implementation reduces overhead on many instances, yet the search space can still remain too large on some graphs if seed-level acceleration is absent.

Second, the comparison with the full optimization pipeline evaluates the combined effect of improved data structures, order-bound pruning, edge-bound pruning, and EBBkC-assisted Turán seeding. Under this configuration, DensePCE becomes faster on every tested dataset. This makes it reasonable to conclude that the largest performance gains come from the interaction between efficient state maintenance and stronger pruning or initialization strategies.

Among the tested graphs, order-bound based early termination was triggered only on `delaunay_n23` and `Protein`. This shows that some pruning rules may be highly effective on specific graph families while having limited influence on others. Therefore, the results also suggest that no single pruning mechanism fully explains the final speedup; rather, performance depends on how multiple mechanisms align with the structural properties of each graph.

### 4.3.3 Practical Significance

The results demonstrate that DensePCE preserves exactness while offering substantially improved runtime in the fully optimized setting. This is important because the work does not change the solution semantics of pseudo-clique enumeration; instead, it shows that careful engineering decisions can significantly improve the practicality

of an exact algorithm.

The use of EBBkC for enumerating the required  $R$ -clique seeds is particularly meaningful in this regard. By starting reverse search from mandatory clique seeds instead of from many shallow states, DensePCE avoids a large amount of overhead near the upper levels of the search tree. At the same time, CSR-based adjacency access and intrusive bucket maintenance reduce repeated costs during recursive expansion and rollback.

Overall, the discussion supports the conclusion that DensePCE achieves its performance advantage through a balanced combination of systems-level optimization and theory-guided pruning. The chapter therefore demonstrates that exact pseudo-clique enumeration can be made noticeably more efficient in practice without sacrificing correctness.



# Chapter 5 Conclusions

## 5.1 Summary

In this project, we designed, implemented, and evaluated **DensePCE**, an optimized exact enumerator for maximal  $(\ell, \theta)$ -pseudo-cliques that targets implementation-level efficiency while preserving the correctness guarantees of reverse-search enumeration. The project was motivated by the observation that existing exact enumerators—most notably FPCE—achieve substantial search-space reduction through structural pruning, yet their practical runtime remains constrained by per-recursion overhead arising from general-purpose data structures and repeated candidate processing.

To address this, DensePCE introduced four baseline technical contributions. First, we replaced adjacency-list graph storage with a sorted *Compressed Sparse Row* (CSR) format, yielding cache-friendly sequential neighborhood access and efficient adjacency tests. Second, we designed *intrusive bucket-based degree lists* that maintain per-vertex degree-to- $P$  counters and support  $O(1)$  bucket moves during vertex addition and removal. Third, we integrated all three FPCE pruning techniques—the degeneracy-based order bound, the core-aware edge bound, and Turán-based clique-existence filtering—into this optimized engine. Fourth, we operationalized the Turán property as an active *R-clique seeding* strategy by streaming exact  $R$ -cliques from the EBBkC  $k$ -clique lister and using each clique as a search seed.

On top of this optimized baseline, this work contributed two theorem-backed refinements for reverse-search candidate handling. The first is *Candidate Uncle Inheri-*

*tance*, which shows that under a sufficient gate the feasible candidate pool of a state can be inherited from the parent side instead of being rebuilt from the whole outside boundary. The second is the *Outside-Gap theorem*, which proves that when the maximum outside degree into the current pseudo-clique is strictly smaller than the minimum inside degree, the feasible pool already equals the exact child set. Together, these results reduce candidate-generation overhead while preserving exactly the same search behavior.

We evaluated DensePCE on ten real-world benchmark graphs under the parameter setting  $\ell = 10, \theta = 0.9$ . The experimental results confirm that DensePCE produces exactly the same set of maximal pseudo-cliques as FPCE, validating correctness. In terms of runtime, DensePCE consistently outperforms FPCE across all tested graphs when all optimizations are enabled, with per-instance speedups ranging from  $1.37\times$  to  $43.6\times$ . Additional ablation experiments show that Candidate Uncle Inheritance provides modest but meaningful gains by reducing candidate-pool construction cost, while the Outside-Gap theorem further improves runtime by removing some residual child-validation overhead without affecting memory usage or search-space size.

Overall, this project demonstrates that careful attention to data layout, memory hierarchy, seed selection, and candidate-generation logic constitutes a dimension of optimization that is complementary to—and composable with—structural pruning. The resulting system pushes the practical frontier of exact pseudo-clique enumeration on large sparse graphs while also contributing two simple but useful theorem-backed refinements to the reverse-search pipeline.

## 5.2 Limitations

Despite the improvements achieved, DensePCE has several limitations that should be acknowledged:

1. **Single-threaded execution.** The current DensePCE implementation is entirely single-threaded. The reverse-search recursion tree contains independent sub-

trees that are, in principle, amenable to parallel exploration. However, the shared incremental state requires careful synchronization or state-copying strategies that have not been implemented.

2. **Restriction to simple undirected graphs.** DensePCE assumes a simple, unweighted, undirected graph model. Extending the pseudo-clique definition and the associated pruning bounds to directed, weighted, or temporal graphs remains outside the current scope.
3. **Overhead of EBBkC seeding for small  $R$ .** When the derived clique size  $R$  is small, the number of  $R$ -cliques in the  $(R-1)$ -core can be very large, and the cost of enumerating all seeds may exceed the savings from skipping upper recursion levels. DensePCE does not yet include an adaptive mechanism for deciding when seeding is beneficial.
4. **Memory consumption on very large graphs.** Although CSR is memory-efficient for graph storage, the auxiliary state for exact reverse-search enumeration still requires  $O(n)$  space. For extremely large graphs, this may exceed available memory.
5. **Limited parameter exploration in evaluation.** Our experiments used a single parameter setting ( $\ell = 10$ ,  $\theta = 0.9$ ) across all graphs. While this enables controlled comparison, it does not fully characterize the behavior of the algorithm over the entire  $(\ell, \theta)$  space.
6. **Modest impact of the theorem-backed candidate refinements.** Candidate Uncle Inheritance and the Outside-Gap theorem reduce overhead but do not directly reduce the number of visited recursive states. Consequently, their empirical gains are necessarily more modest than those of large search-space pruning rules. The Outside-Gap theorem, in particular, acts as a fast exact path only when a specific structural separation between inside and outside degrees holds, and therefore its activation is graph-dependent.

7. **No fully general simplification of child validation.** Our investigation suggests that the child-validity rule of the PCE reverse-search framework cannot, in general, be reduced to a purely local degree-based gate. The Outside-Gap theorem therefore represents a useful but special-case simplification rather than a universal replacement for the child characterization.

## 5.3 Future Improvement

Several directions for future work can extend and strengthen the contributions of this project:

1. **Weighted and directed graph extensions.** Generalizing the density definition to incorporate edge weights ( requiring the weighted density to exceed  $\theta$ ) or extending to directed graphs ( requiring both in-density and out-density constraints) would enable DensePCE to handle a wider class of networks. The pruning bounds would need to be rederived under these generalized models.
2. **Adaptive pruning strategy selection.** Currently, all pruning techniques are either globally enabled or disabled. A more sophisticated approach would dynamically enable or disable individual pruning checks based on intermediate search statistics—for example, disabling the edge-bound check when it rarely triggers, or switching from seeded to unseeded search when  $R$  is too small for seeding to be beneficial. Such adaptive strategies could reduce the overhead of pruning checks that provide marginal benefit on specific graph instances.
3. **Hybrid seed strategies.** Beyond  $R$ -clique seeding, other dense substructures (dense  $k$ -plexes, high-connectivity subgraphs) could serve as alternative or complementary seeds. Exploring a portfolio of seed types and selecting the most effective one based on graph properties could further reduce the explored search space.
4. **Bridge to approximate enumeration.** While DensePCE is an exact enumerator,

many practical applications may tolerate a controlled amount of incompleteness in exchange for dramatically faster execution. Developing a mode that relaxes the completeness guarantee—for example, by sampling a representative subset of maximal pseudo-cliques or by terminating search branches probabilistically—could extend DensePCE’s utility to interactive and time-constrained analysis scenarios.

5. **Distributed-memory scaling.** For graphs that exceed single-machine memory (web-scale social networks), partitioning the graph across multiple machines and coordinating the reverse-search traversal in a distributed fashion would enable DensePCE to scale to datasets with billions of edges. This would require addressing challenges in cross-partition adjacency queries, seed distribution, and duplicate elimination.

These directions highlight the potential for continued advances in exact dense subgraph enumeration through a combination of parallelism, richer graph models, adaptive algorithmic strategies, and systems-level engineering.

# References

- [1] M. E. Newman, "Network structure from rich but noisy data," *Nature Physics*, vol. 14, no. 6, pp. 542–545, 2018.
- [2] J. Zhang and K. Huang, "Normalized lmqcm: An algorithm for detecting weak quasi-cliques in weighted graph with applications in gene co-expression module discovery in cancers," *Cancer Informatics*, vol. 13, pp. CIT-S14 021, 2016.
- [3] C. Bron and J. Kerbosch, "Algorithm 457: Finding all cliques of an undirected graph," *Communications of the ACM*, vol. 16, no. 9, pp. 575–577, 1973.
- [4] D. Eppstein, M. Löffler, and D. Strash, "Listing all maximal cliques in sparse graphs in near-optimal time," *Proceedings of the 21st International Symposium on Algorithms and Computation (ISAAC 2010)*, vol. 6506, pp. 403–414, 2010.
- [5] D. Avis and K. Fukuda, "Reverse search for enumeration," *Discrete Applied Mathematics*, vol. 65, no. 1-3, pp. 21–46, 1996.
- [6] T. Uno, "An efficient algorithm for enumerating pseudo cliques," in *Proceedings of the 21st International Symposium on Algorithms and Computation (ISAAC)*. Springer, 2010, pp. 402–414.
- [7] A. Rahman, K. Roy, R. Maliha, and T. F. Chowdhury, "A fast exact algorithm to enumerate maximal pseudo-cliques in large sparse graphs," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 2479–2490.

- [8] D. W. Matula and L. L. Beck, "Smallest-last ordering and clustering and graph coloring algorithms," *Journal of the ACM*, vol. 30, no. 3, pp. 417–427, July 1983.
- [9] V. Batagelj and M. Zaveršnik, "An  $O(m)$  algorithm for cores decomposition of networks," arXiv:cs.DS/0310049, 2003.
- [10] P. Turán, "On an extremal problem in graph theory," *Matematikai és Fizikai Lapok*, vol. 48, pp. 436–452, 1941.
- [11] M. Danisch, O. Balalau, and M. Sozio, "Listing k-cliques in sparse real-world graphs," in *Proceedings of the 2018 World Wide Web Conference*, ser. WWW '18. ACM, 2018, pp. 589–598.
- [12] J. Abello, M. G. Resende, and S. Sudarsky, "Massive quasi-clique detection," in *Latin American symposium on theoretical informatics*. Springer, 2002, pp. 598–612.
- [13] C. Komusiewicz, M. Sorge, and K. Stahl, "Finding connected subgraphs of fixed minimum density: Implementation and experiments," in *International Symposium on Experimental Algorithms*. Springer, 2015, pp. 82–93.
- [14] J. Long and C. Hartman, "Odes: an overlapping dense sub-graph algorithm," *Bioinformatics*, vol. 26, no. 21, pp. 2788–2789, 2010.
- [15] G. Liu and L. Wong, "Effective pruning techniques for mining quasi-cliques," in *Joint European conference on machine learning and knowledge discovery in databases*. Springer, 2008, pp. 33–49.
- [16] H. Zhai, M. Haraguchi, Y. Okubo, and E. Tomita, "A fast and complete enumeration of pseudo-cliques for large graphs," in *Advances in Knowledge Discovery and Data Mining: 20th Pacific-Asia Conference, PAKDD 2016, Auckland, New Zealand, April 19-22, 2016, Proceedings, Part I* 20. Springer, 2016, pp. 423–435.
- [17] L. Chang, "Efficient maximum clique computation over large sparse graphs," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. ACM, 2019, pp. 599–608.

- [18] K. Wang, K. Yu, and C. Long, “Efficient  $k$ -clique listing: An edge-oriented branching strategy,” *arXiv preprint arXiv:2311.13798*, 2023.